

分布式流处理框架 Flink

Grissom | 2025年12月

目录>

CONTENTS

- 1 Flink 简介
- 2 Flink 原理
- 3 Flink 基础使用



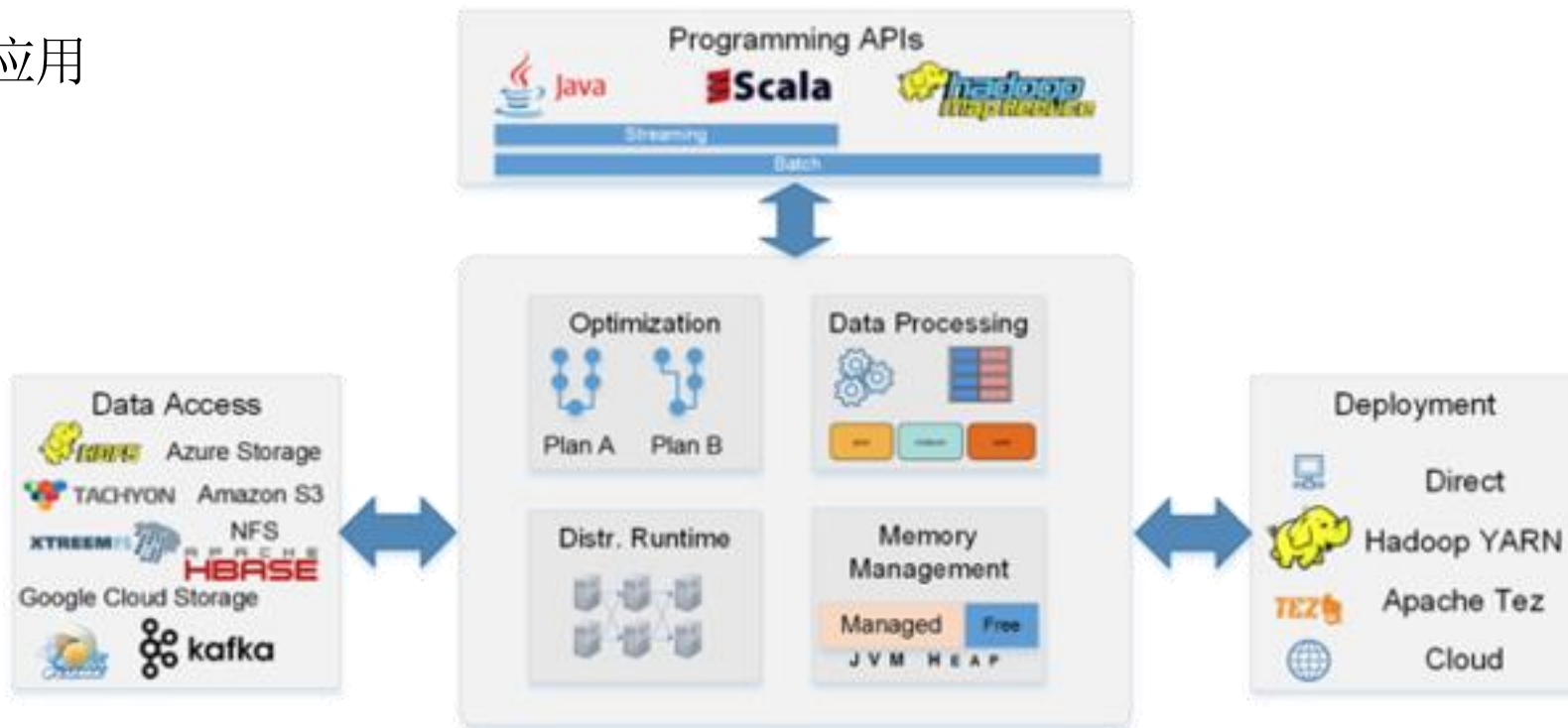
1

chapter

Flink 简介

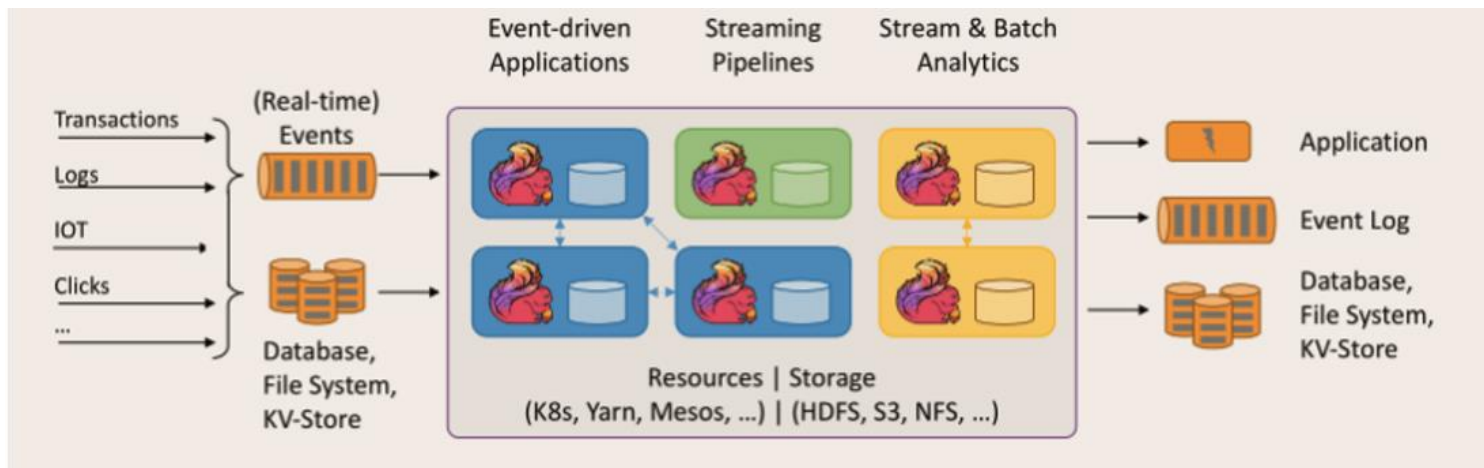
➤ 什么是Flink

- Apache Flink是一个开源的流处理框架，高吞吐、低延迟，拥有强大的状态管理能力
- 起源于Stratosphere项目，2014年4月提交到Apache软件基金会进行孵化
- 用于处理大规模的数据流，并能以极高的性能运行复杂的事件驱动型应用

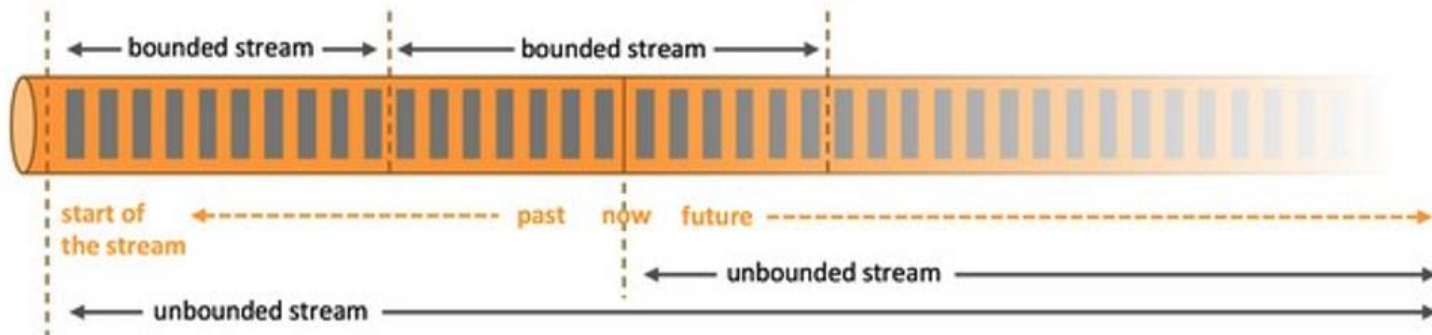


➤ Flink优势:

- 流批一体：允许开发者使用相同的API来处理实时数据和历史数据

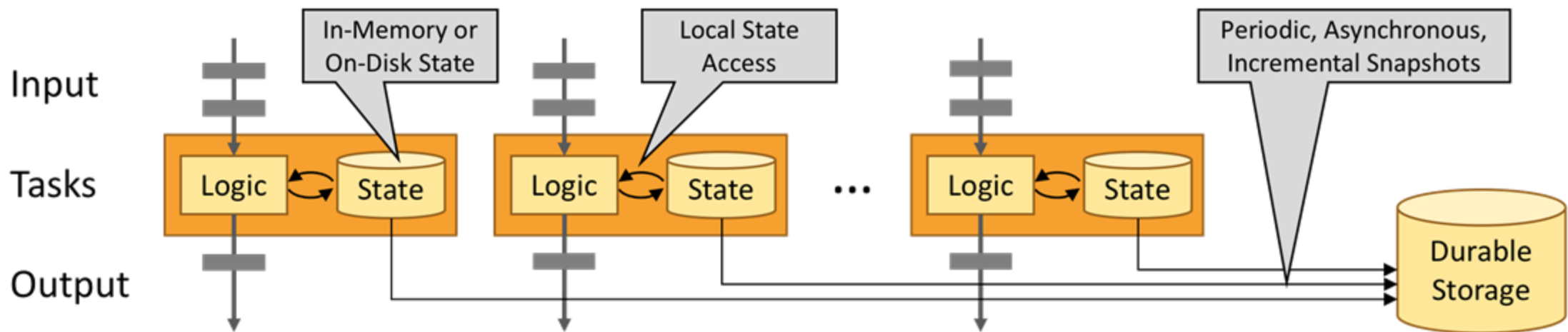


- ◆ 批处理：有界数据流，明确定义了数据的开始和结束，可以在执行计算之前获取到所有数据集进行计算。
- ◆ 流处理：无界数据流，有开始但没有结束，必须连续处理无界数据流，通常要按照特定顺序获取每条数据，从而保证计算结果的完整性。



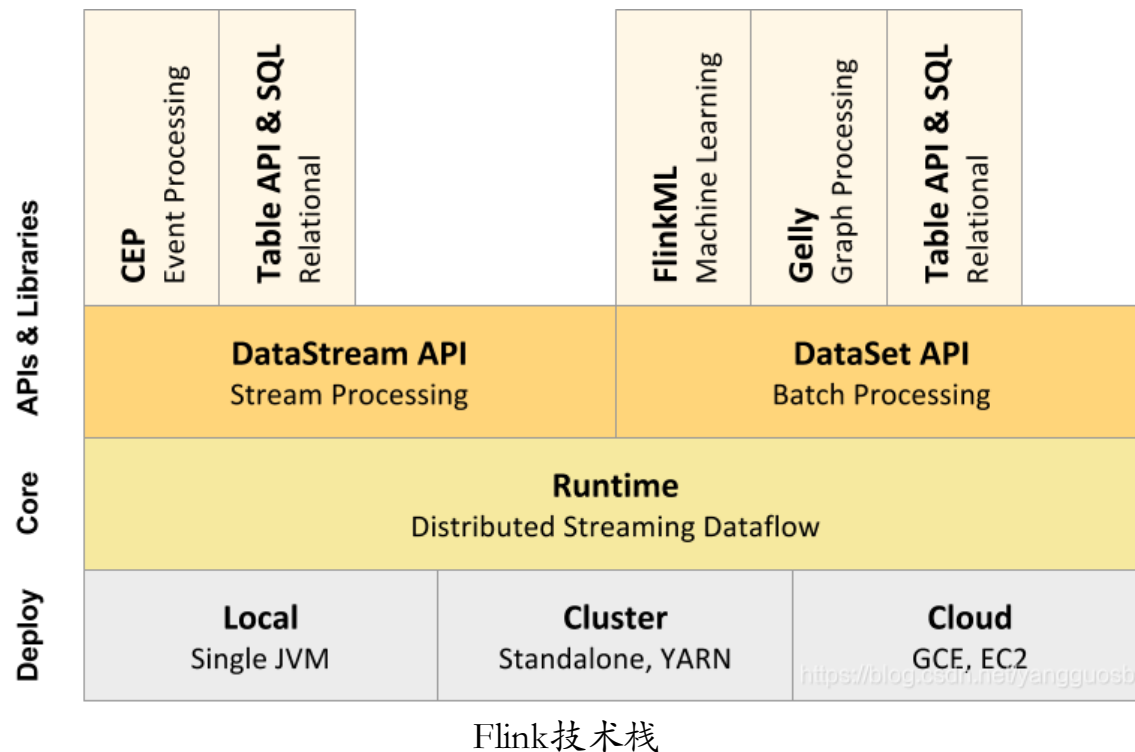
➤ Flink优势:

- 利用内存性能：任务的状态始终保留在内存中，如果状态大小超过可用内存，则会保存在能高效访问的磁盘数据结构中。任务通过访问本地（通常在内存中）状态来进行所有的计算，从而产生非常低的处理延迟。Flink 通过定期和异步地对本地状态进行持久化存储来保证故障场景下精确一次的状态一致性。



➤ Flink优势:

- API 和库: 提供了多种编程语言的API, 以及用于复杂事件处理、机器学习等的库
- 支持事件时间: 在分布式环境中也能保持数据处理的一致性
- 灵活的窗口操作: 内置滚动窗口、滑动窗口和会话窗口, 以支持复杂的时间相关操作
- 丰富的连接器: 提供各种存储系统、消息队列和其他外部系统的连接器, 方便数据的进出
- 容错机制: 提供强大的状态管理功能和容错机制, 能够保证在发生故障时的状态恢复



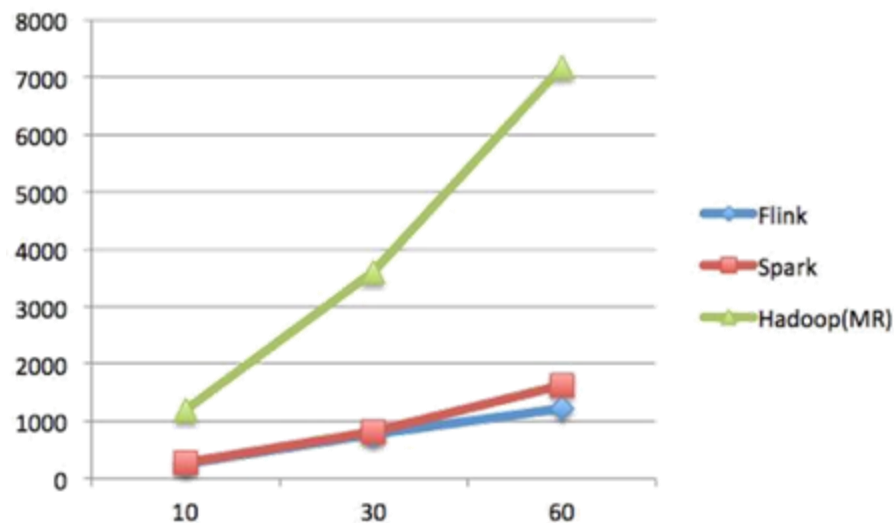
➤ Flink vs SparkStreaming:

• 相同点:

- ① 基于内存的计算框架，因此，都可以获得较好的实时计算性能；
- ② 统一的批处理和流处理API，都支持类似SQL的编程接口；
- ③ 支持很多相同的转换操作，编程都是用函数式编程模式；
- ④ 完善的错误恢复机制；
- ⑤ 支持“精确一次”（exactly once）的语义一致性。

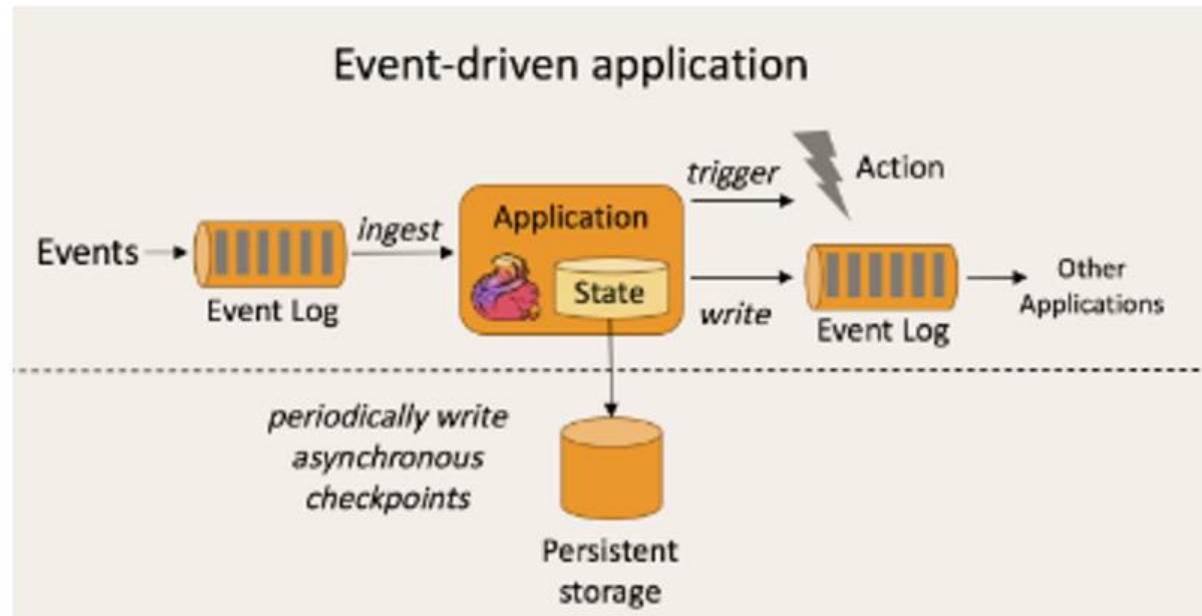
• 区别:

- Spark的技术理念是基于批来模拟流的计算。而Flink则完全相反，它采用的是基于流计算来模拟批计算。从技术发展方向看，用批处理来模拟流计算有一定的技术局限性，并且这个局限性可能很难突破。而Flink基于流计算来模拟批处理，在技术上有更好的扩展性。
- Flink和Spark都支持流计算，二者的区别在于，Flink是一条一条地处理数据，而Spark是基于RDD的小批量处理，所以，Spark在流式处理方面，不可避免地会增加一些延时，实时性没有Flink好。Flink的流计算性能和Storm差不多，可以支持毫秒级的响应，而Spark则只能支持秒级响应。
- 当全部运行在Hadoop YARN之上时，Flink的性能要略好于Spark，因为，Flink支持增量迭代，具有对迭代进行自动优化的功能。



➤ 事件驱动模式 vs 微批模式：

Flink:
(事件驱动)



SparkStreaming:
(微批模式)



➤ Flink应用场景:

- 实时智能推荐
- 复杂事件处理
- 实时欺诈检测
- 实时数仓与 ETL
- 流数据分析
- 实时报表分析



2

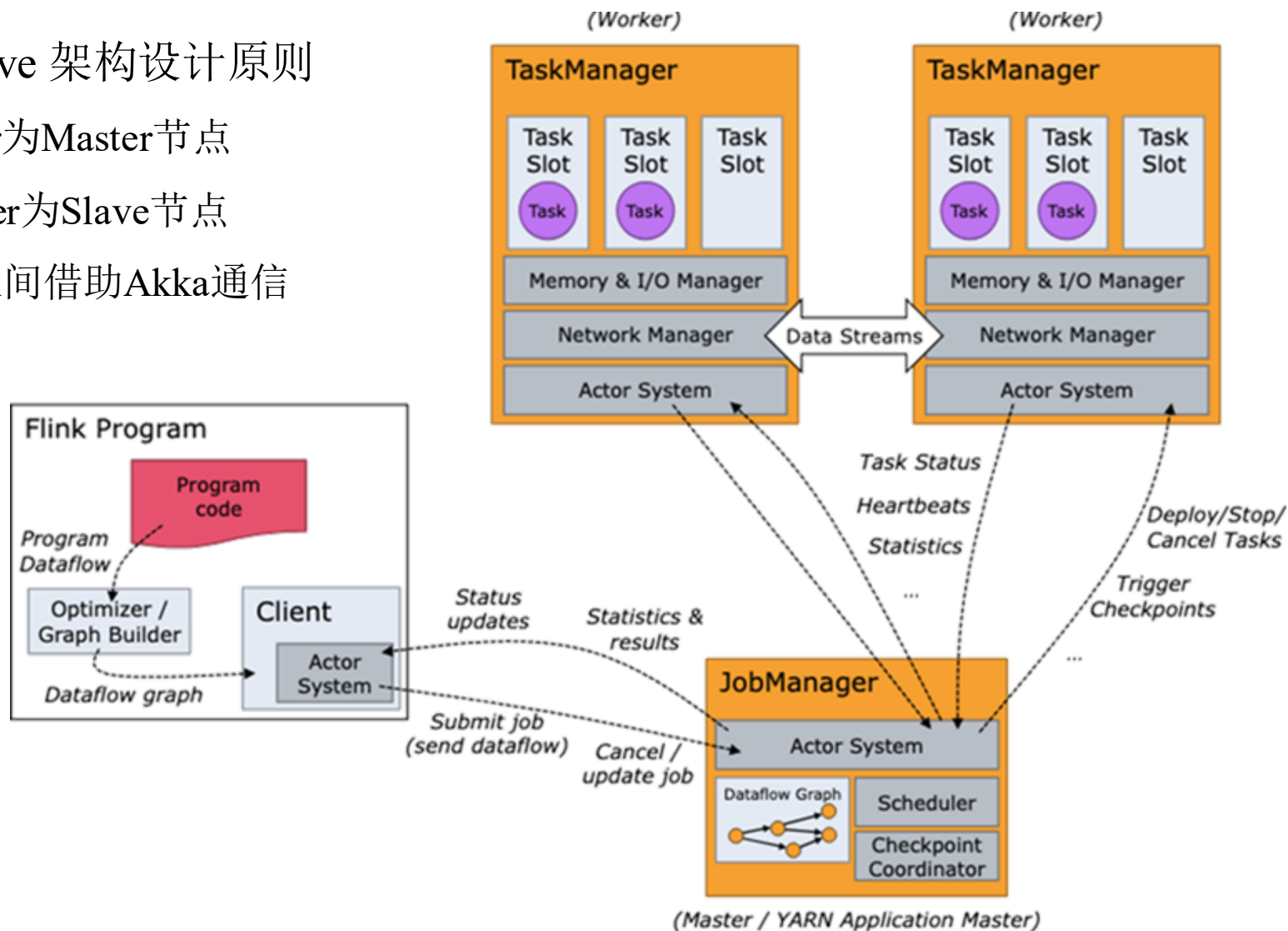
chapter

Flink 原理

- ✓ Flink 运行架构
- ✓ Flink 运行模式

➤ Flink 运行架构

- 遵循 Master-Slave 架构设计原则
 - JobManager为Master节点
 - TaskManager为Slave节点
 - 所有组件之间借助Akka通信



➤ Flink 运行架构

- JobManager

- 主要负责调度 Job 并协调 Task 做 checkpoint，职责上很像 Storm 的 Nimbus。
- 从 Client 处接收到 Job 和 JAR 包等资源后，会生成优化后的执行计划，
- 并以 Task 的单元调度到各个TaskManager 去执行

- TaskManager

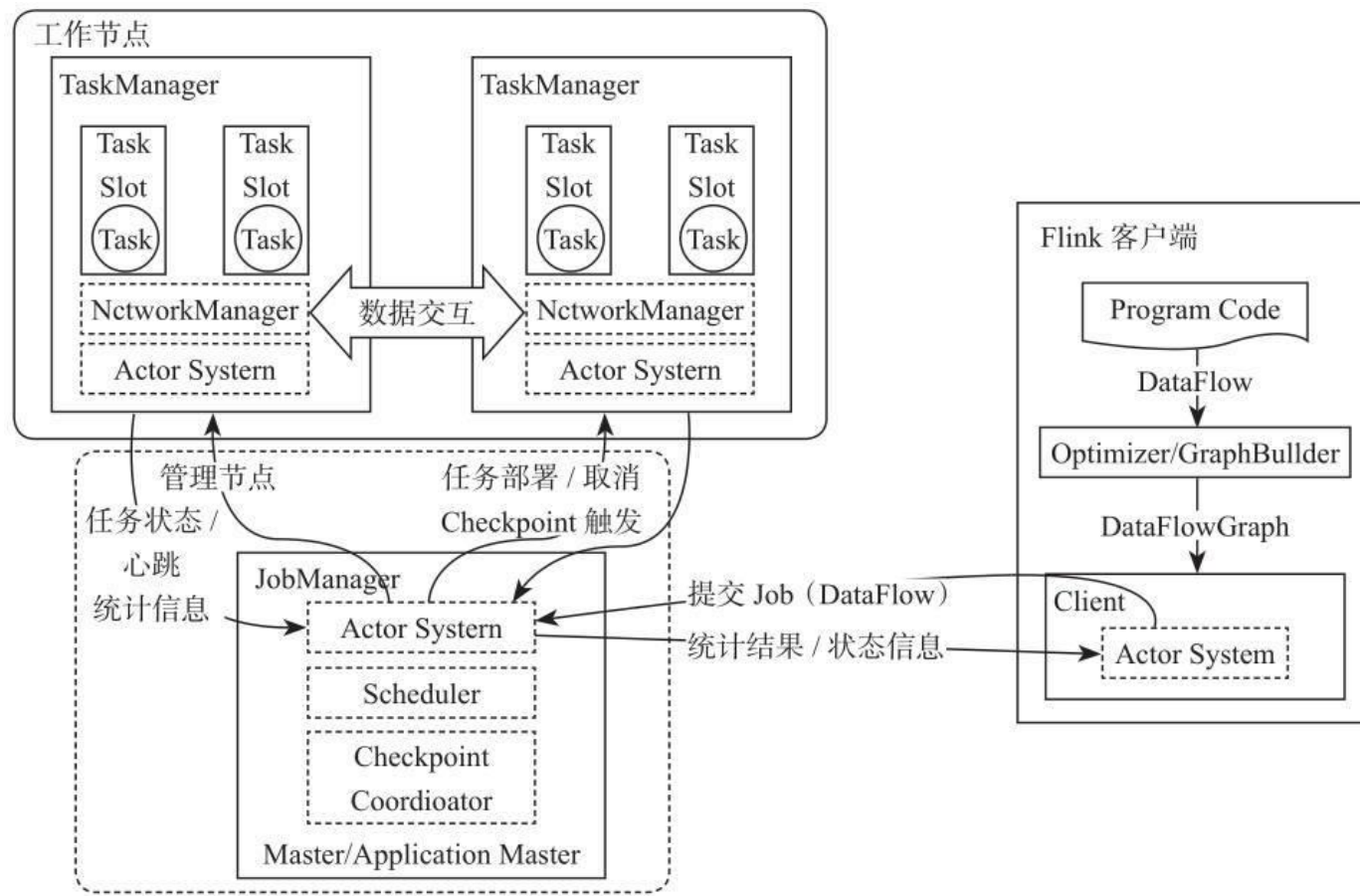
- 在启动的时候就设置好了槽位数（Slot），每个 slot 能启动一个 Task，Task 为线程。
- 从 JobManager 处接收需要部署的 Task，部署启动后，与自己的上游建立 Netty 连接，接收数据并处理。

- Client

- 为提交 Job 的客户端，可以是运行在任何机器上（与 JobManager 环境连通即可）。
- 提交 Job 后，Client 可以结束进程（Streaming的任务），也可以不结束并等待结果返回。

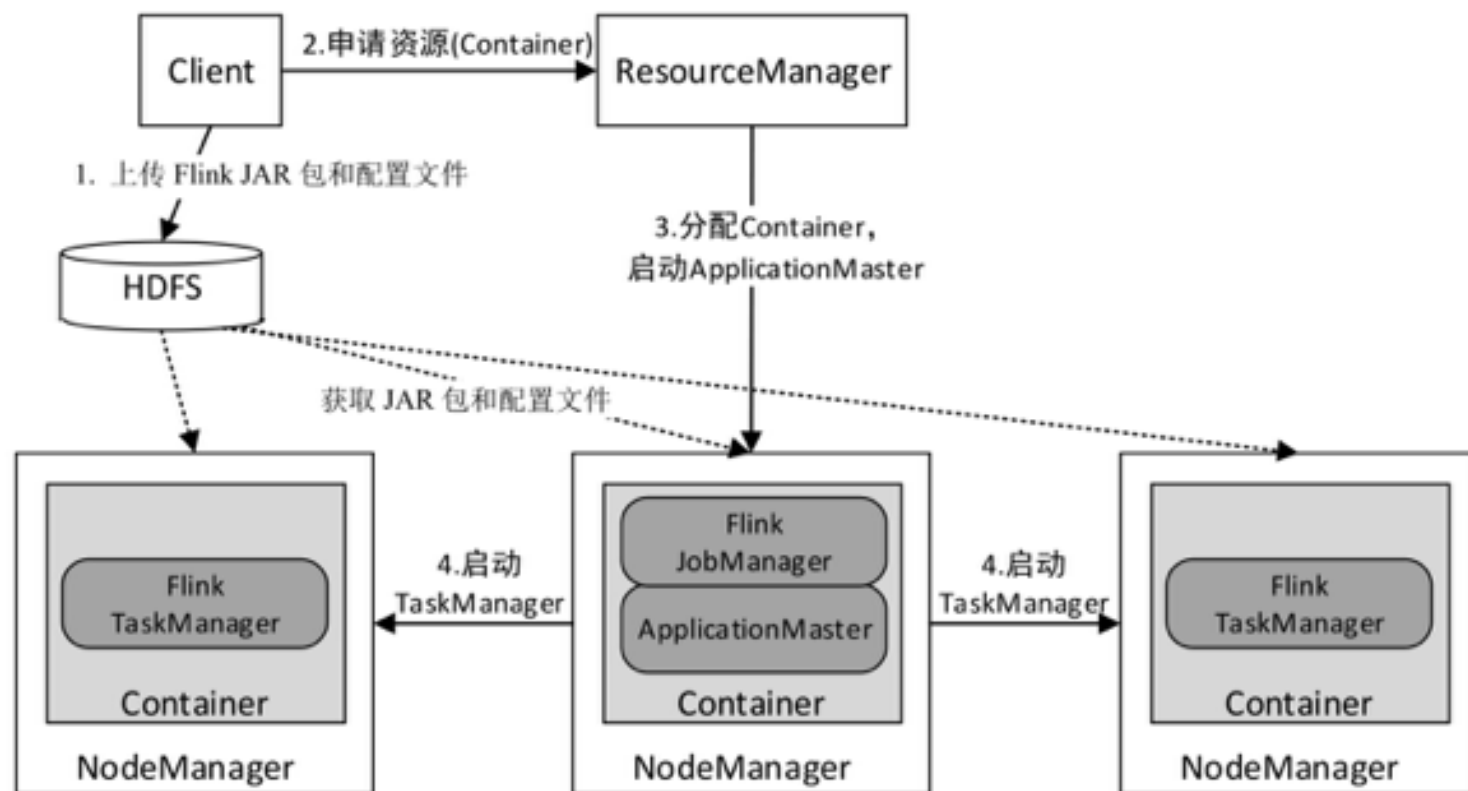
➤ Flink 运行模式

- local
- **Standalone**
- On Yarn



➤ Flink 运行模式

- local
- Standalone
- **On Yarn**





3 chapter

Flink 基础使用

- ✓ Flink 开发模型
- ✓ Flink SQL

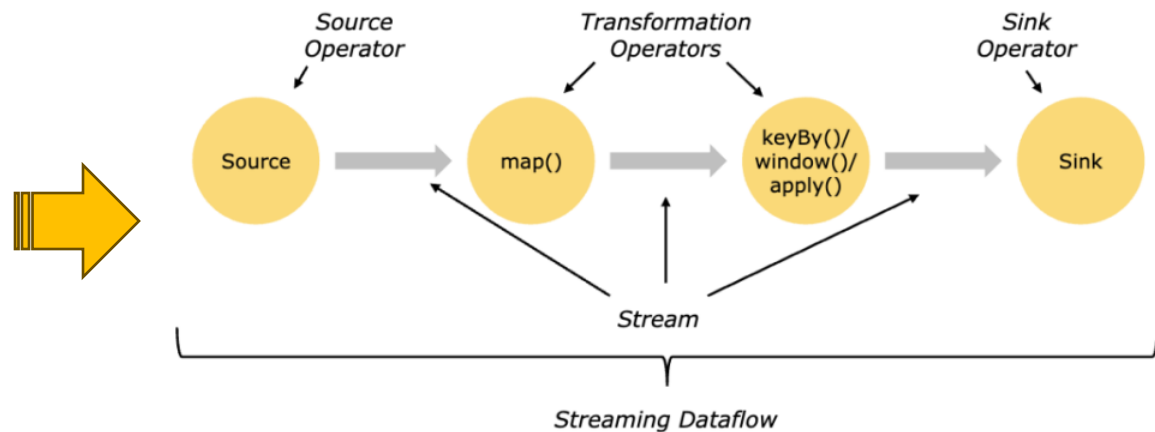
➤ Flink 开发模型

• Flink对数据处理可以抽象为以下三个步骤：

- ① 接受数据，接收一个或者多个数据源（hdfs，kafka等）。
- ② 计算数据，执行若干用户需要的转换算子。
- ③ 输出结果，将转换后的结果输出。

```
DataStream<String> lines = env.addSource(  
    new FlinkKafkaConsumer<> (...));  
DataStream<Event> events = lines.map((line) -> parse(line));  
DataStream<Statistics> stats = events  
    .keyBy(event -> event.id)  
    .timeWindow(Time.seconds(10))  
    .apply(new MyWindowAggregationFunction());  
stats.addSink(new MySink(...));
```

Source
Transformation
Transformation
Sink



➤ Flink 常用算子

- Source: 数据源
 - 本地集合的 source、基于文件的 source、基于网络套接字的 source、自定义 source
- Transformation: 数据转换操作
 - Map / FlatMap / Filter / KeyBy / Reduce / Fold / Aggregations / Window / WindowAll / Union / Window join / Split / Select / Project 等
- Sink: 输出
 - 写入文件、标准输出流（打印）、写入 Socket、自定义的 Sink



➤ 案例①：实时分析

```
// 每隔1秒钟统计最近2秒钟的每个单词出现的次数
object FlinkStream {
  def main(args: Array[String]): Unit = {
    //1、构建流处理的环境
    val env: StreamExecutionEnvironment = StreamExecutionEnvironment.getExecutionEnvironment

    //2、从socket获取数据
    val sourceStream: DataStream[String] = env.socketTextStream("node01",9999)

    //3、对数据进行处理
    val result: DataStream[(String, Int)] = sourceStream
      .flatMap(x => x.split(" ")) //按照空格切分
      .map(x => (x, 1))           //每个单词计为1d
      .keyBy(0)                   //按照下标为0的单词进行分组
      .sum(1)                     //按照下标为1累加相同单词出现的次数

    result.print()               //4、对数据进行打印
    env.execute("FlinkStream") //5、开启任务
  }
}
```

➤ 案例②：离线分析

```
// Scala开发Flink的批处理程序
object FlinkFileCount {
  def main(args: Array[String]): Unit = {
    //1、构建Flink的批处理环境
    val env: ExecutionEnvironment = ExecutionEnvironment.getExecutionEnvironment

    //2、读取数据文件
    val fileDataSet: DataSet[String] = env.readTextFile("d:\\words.txt")

    //3、对数据进行处理
    val resultDataSet: AggregateDataSet[(String, Int)] = fileDataSet
      .flatMap(x=> x.split(" "))
      .map(x=>(x,1))
      .groupBy(0)
      .sum(1)

    resultDataSet.print() // 4、打印结果
    resultDataSet.writeAsText("d:\\result") // 5、保存结果到文件
    env.execute("FlinkFileCount")
  }
}
```

➤ Flink SQL

- 基于Flink core，使用SQL语义方便快捷的进行结构化数据处理的上层库，允许用户以SQL语句的形式对有界（批处理）和无界（流处理）数据进行查询和分析。

- 特点：

- 统一的批流处理：

Flink SQL对批处理和流处理提供了统一的编程模型。

使用相同的SQL语义，可以处理静态数据集（批处理）以及实时流的动态数据（流处理）

- 声明式编程：

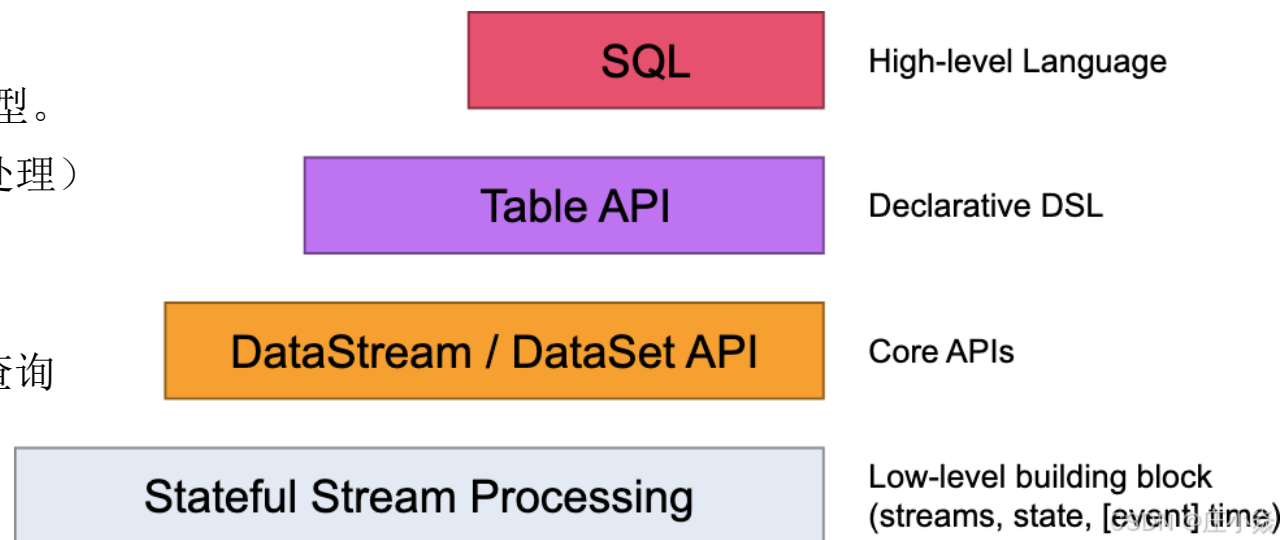
用户只需要通过SQL语句描述所需的数据转换和查询逻辑，而无需关心底层的实现细节。

- 与标准SQL高度兼容：

Flink SQL支持大部分标准的SQL语法。

- 可扩展性：

Flink SQL可以与Flink的其他功能模块（如用户自定义函数、连接器等）结合使用，满足复杂的业务需求。



➤ 动态表 & 持续查询

- Flink采用了**动态表**和**持续查询**来解决流处理（实时SQL处理）的问题。
- 动态表是Flink在Table API和SQL中的核心概念，它为流数据处理提供了表和SQL支持。
- 与静态表相比，动态表随时间而变化，当有新的数据到达，就会在动态表中加入新的一行。
可以像静态表一样查询动态表，只不过查询动态表需要产生连续查询。
- 连续查询永远不会终止，每次有新的数据到达，都会触发查询，会重新生成动态表作为结果表。查询不断更新其（动态）结果表以反映其（动态）输入表的更改。
- 特点：
 - 数据源的输入是持续的
 - 查询过程是持续的
 - 结果输出也是持续的

➤ 动态表 & 持续查询

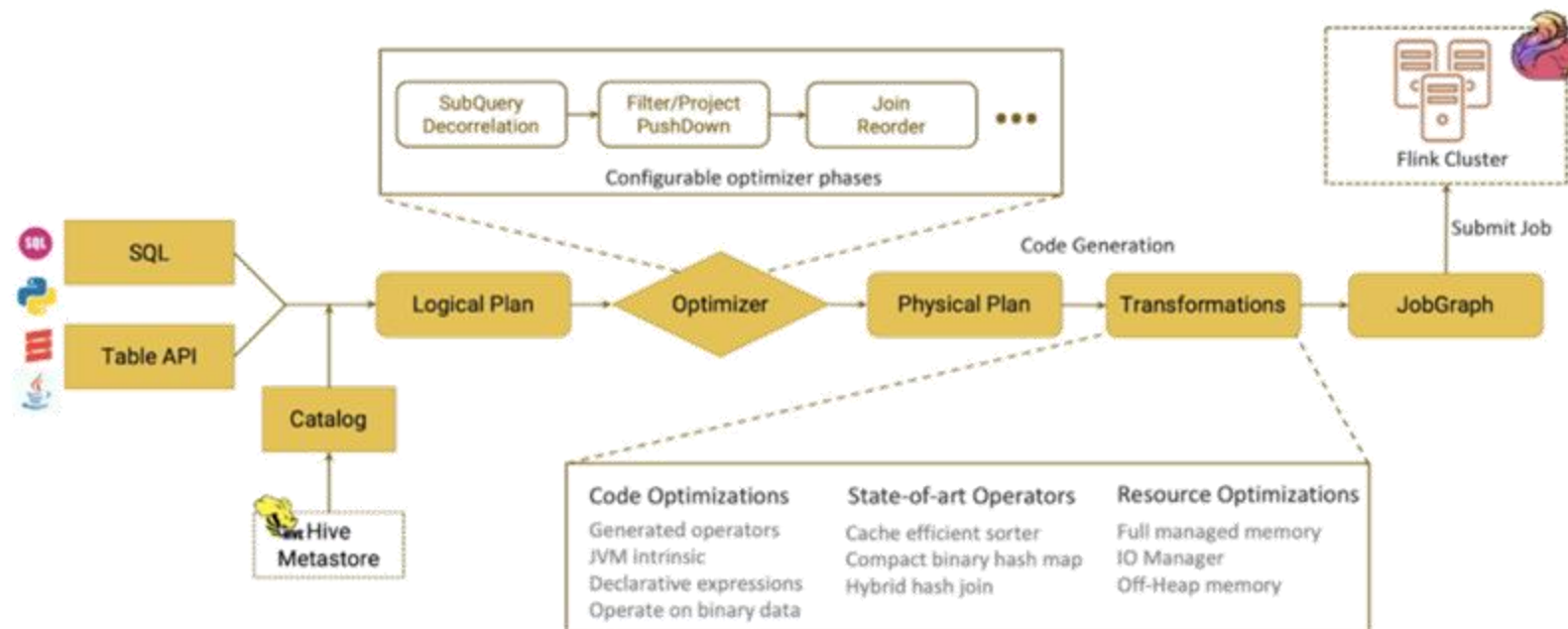
• 持续查询步骤:

- ① 流被转换为动态表
- ② 对动态表进行持续查询，生成新的动态表
- ③ 生成的动态表被转换成流



传统SQL	流处理
关系（或表）是有界的（多）元组集。	流是元组的无限序列。
对批处理数据（例如，关系数据库中的表）执行的查询可以访问完整的输入数据。	流式查询在启动时无法访问所有数据，必须“等待”数据流入。
批量查询在生成固定大小的结果后终止。	流式查询根据收到的记录不断更新其结果，并且永远不会完成。

➤ Flink SQL 执行流程



- ① 将 SQL文本 / TableAPI 代码转化为逻辑执行计划（Logical Plan）
- ② Logical Plan 通过优化器优化为物理执行计划（Physical Plan）
- ③ 通过代码生成技术生成 Transformations 后进一步编译为可执行的 JobGraph 提交运行

➤ 案例：Table API

```
// 每隔10秒，统计不同商品的浏览量
def main(args: Array[String]): Unit = {
  val env: StreamExecutionEnvironment = StreamExecutionEnvironment.getExecutionEnvironment
  val dstream: DataStream[String] = env.addSource(MyKafkaUtil.getConsumer("GMALL_STARTUP"))
  val startupLogDstream: DataStream[StartupLog] = dstream.map { jsonString => JSON.parseObject(jsonString, classOf[StartupLog]) }
  ... ..

  //把数据流转化成Table
  val startupTable: Table = tableEnv.fromDataStream(xxdstream, 'uid,'uid,'appid,'aid,'aid,'area, 'begintime,'ts.rowtime)

  // 通过table api 进行操作
  // 1.roupby; 2.window (Tumble固定大小且不重叠的时间窗口)
  val resultTable: Table = startupTable.window(Tumble over 10000.millis on 'ts as 'tt).groupBy(aid,'tt ).select( aid, 'uid.count)

  //把Table转化成数据流
  val resultDstream: DataStream[(Boolean, (String, Long))] = resultSQLTable.toRetractStream[(String,Long)]

  resultDstream.filter(_._1).print()
  env.execute()
}
```

➤ 案例：Table SQL

```
// 每隔10秒，统计不同商品的浏览量
def main(args: Array[String]): Unit = {
  val env: StreamExecutionEnvironment = StreamExecutionEnvironment.getExecutionEnvironment
  val dstream: DataStream[String] = env.addSource(MyKafkaUtil.getConsumer("GMALL_STARTUP"))
  val startupLogDstream: DataStream[StartupLog] = dstream.map { jsonString => JSON.parseObject(jsonString, classOf[StartupLog]) }
  ... ..

  //把数据流转化成Table
  val startupTable: Table = tableEnv.fromDataStream(xxdstream, 'uid,'uid,'appid,'aid,'aid,'area, 'begintime,'ts.rowtime)

  // 通过 SQL 进行操作
  // Flink SQL 提供两个执行入口：StreamTableEnvironment.executeSql(sql)，以及StreamTableEnvironment.sqlQuery(sql)
  val resultSQLTable : Table =
    tableEnv.sqlQuery( "select aid ,count(uid)  from "+startupTable+" group by aid  ,Tumble(ts,interval '10' SECOND )")

  //把Table转化成数据流
  val resultDstream: DataStream[(Boolean, (String, Long))] = resultSQLTable.toRetractStream[(String,Long)]

  resultDstream.filter(_._1).print()
  env.execute()
}
```

➤ 案例: Table SQL

```
--首先创建一个表 Orders 来存储订单数据
CREATE TABLE Orders (
  order_id INT,
  user_id INT,
  order_amount DOUBLE,
  order_time TIMESTAMP(3)
) WITH (
  'connector' = 'kafka', -- 使用 Kafka 作为数据源
  'topic' = 'orders_topic', -- Kafka 的主题
  'properties.bootstrap.servers' = 'localhost:9092', -- Kafka 服务器地址
  'properties.group.id' = 'flink-group', -- 消费者组 ID
  'scan.startup.mode' = 'latest-offset', -- 从最新的 offset 开始读取
  'format' = 'json' -- 数据格式为 JSON
);
--编写一个 SQL 查询来选择所有订单，并过滤出订单金额大于 150 的记录
SELECT * FROM Orders WHERE order_amount > 150;
--进行一个聚合查询，计算每个用户的总订单金额
SELECT user_id, SUM(order_amount) AS total_amount FROM Orders GROUP BY user_id;
--结果输出
CREATE TABLE ResultTable (
  user_id INT,
  total_amount DOUBLE) WITH ('connector' = '...', -- 选择输出连接器...);

INSERT INTO ResultTable
SELECT user_id, SUM(order_amount) AS total_amount FROM Orders GROUP BY user_id;
```



Q&A

TRANSWARP
星环科技